

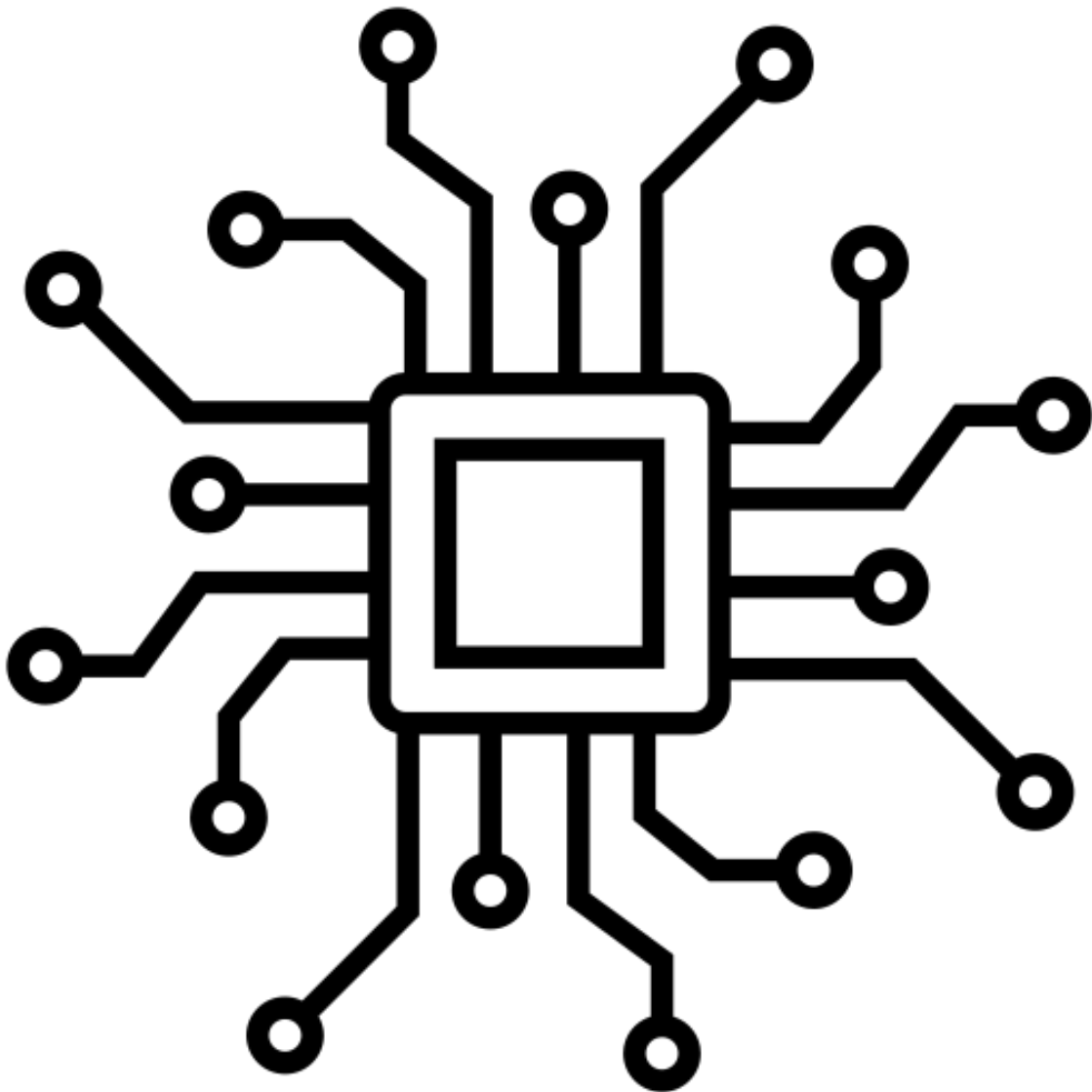
Ψηφιακά Συστήματα ΗΥ σε Χαμηλά Επίπεδα Λογικής II

Σχεδίαση Floating Point Adder IEEE-754

Ονοματεπώνυμο: Παναγιώτης Λεκός

Email: panaleko@ece.auth.gr

AEM: 10347



1. Εισαγωγή

Το παρόν έγγραφο αποτελεί την τεχνική αναφορά της σχεδίασης και επαλήθευσης ενός αθροιστή κινητής υποδιαστολής απλής ακρίβειας στα πλαίσια της Εργασίας του μαθήματος “Ψηφιακά Συστήματα HW σε Χαμηλά Επίπεδα Λογικής II”. Το κύκλωμα σχεδιάστηκε και επαληθεύτηκε σύμφωνα με το εγχειρίδιο εργαστηρίου του μαθήματος και βασίζεται στο πρότυπο IEEE-754. Πιο συγκεκριμένα, η σχεδίαση υλοποιεί μερική υποστήριξη του προτύπου, κατά την οποία οι denormal αριθμοί αντιμετωπίζονται ως μηδέν ($\pm Zero$) και οι αριθμοί NaN ως άπειρο ($\pm Infinity$). Για τους σκοπούς αυτούς χρησιμοποιήθηκε η γλώσσα περιγραφής και επαλήθευσης υλικού SystemVerilog και το εργαλείο προσομοιώσεων Questa Intel Starter FPGA Edition-64. Παρακάτω θα περιγραφεί η αρχιτεκτονική όλων των modules του κυκλώματος, καθώς και η διαδικασία επαλήθευσής του.

2. Σχεδίαση και Αρχιτεκτονική Υλικού

Στην παρούσα ενότητα παρουσιάζεται η αρχιτεκτονική των επιμέρους υπομονάδων (sub-modules) του αθροιστή κινητής υποδιαστολής. Η σχεδίαση ακολουθεί ιεραρχική δομή, με ένα κεντρικό κύκλωμα το οποίο είναι υπεύθυνο για την σωστή συνδεσμολογία όλων των modules.

2.1 Main Module και Wrapper

Τα δύο αυτά modules υλοποιήθηκαν στα αρχεία *fp_adder.sv* και *fp_adder_top.sv*. Ο wrapper περιλαμβάνει την συνδυαστική λογική του main module, συνδέοντας τις εισόδους και εξόδους του με καταχωρητές. Έτσι εισάγεται ακολουθιακή λογική στο κύκλωμα με αποτέλεσμα τα δεδομένα εισόδου να διαμορφώνουν την έξοδο με καθυστέρηση 2 κύκλων ρολογιού. Επίσης υλοποιείται η λειτουργία του ασύγχρονου, χαμηλά ενεργού reset.

Από την άλλη, το main module περιέχει την καθαρά συνδυαστική λογική του αθροιστή. Παίρνει ως εισόδους τους δύο αριθμούς κινητής υποδιαστολής (a , b) των 32 bit που πρόκειται να προστεθούν και τον τύπο της στρογγυλοποίησης ($round$) του αποτελέσματος. Στη έξοδο βγάζει το αποτέλεσμα των 32 bit ($result$) και 8 bits ($status$) τα οποία δίνουν πληροφορίες για αυτό, όπως το αν βρίσκεται σε κάποια από τις ειδικές περιπτώσεις που ορίζονται στο πρότυπο ή αν υπήρξε overflow/underflow κατά τη διάρκεια υπολογισμού, επηρεάζοντας την ακρίβεια του αποτελέσματος. Για καλύτερη διαχείριση ορίστηκε η δομή struct *float_t* η οποία χωρίζει έναν 32 bit αριθμό σε *sign* (1 bit), *exp* (8 bit), και *mant* (23 bit). Μέσω εσωτερικών σημάτων συνδέονται κατάλληλα όλα τα sub-modules ώστε να παραχθούν οι έξοδοι. Εκτός αυτών πραγματοποιούνται ακόμα δύο λειτουργίες.

Η πρώτη είναι ο υπολογισμός του πρόσημου του αποτελέσματος (z_sign). Στο εσωτερικό ενός συνδυαστικού μπλοκ, υπολογίζεται με βάση τους δύο τελεστές. Εάν έχουν το ίδιο πρόσημο τότε αυτό διατηρείται και στο αποτέλεσμα. Διαφορετικά, συγκρίνονται οι εκθέτες και διατηρείται το πρόσημο του μεγαλύτερου. Αν οι εκθέτες είναι ίσοι, συγκρίνονται παρόμοια τα κλασματικά μέρη και αν και αυτά είναι ίσα, τότε το πρόσημο ορίζεται αυθαίρετα θετικό.

Η επόμενη λειτουργία είναι το Post-Round Normalization, όπου μέσα σε ένα συνδυαστικό μπλοκ ελέγχεται εάν η στρογγυλοποίηση στο rounding module προκάλεσε overflow στο κλασματικό μέρος του αποτελέσματος. Αν αυτό συνέβη τότε τροποποιεί κατάλληλα και το κλασματικό και το εκθετικό μέρος, ώστε να συμβαδίζει με το πρότυπο. Επειδή αυτή η αλλαγή μπορεί να προκαλέσει overflow/underflow στον εκθέτη υπάρχουν τα αντίστοιχα flags που δείχνουν αν έγινε αυτό, ώστε να τα διαχειριστεί κατάλληλα το exception module.

2.2 Υπολογισμός Εκθέτη

Αυτό το module υλοποιήθηκε στο αρχείο *exponent_calc.sv* και είναι υπεύθυνο για τον υπολογισμό του εκθετικού μέρους του αποτελέσματος. Στις εισόδους του παίρνει τα 8 bit εκθετικά μέρη των δύο τελεστών (Ea , Eb) και συγκρίνοντάς τα δίνει στις εξόδους του τον μεγαλύτερο εκθέτη (max_exp), την απόλυτη διαφορά τους (exp_diff), και ένα σήμα του ενός bit που υποδεικνύει ποιο από τα δύο είναι μεγαλύτερο ($sign_exp_diff$). Στην περίπτωση που είναι ίσα διαλέγεται αυθαίρετα ο εκθέτης της εισόδου a ως ο μεγαλύτερος.

2.3 Υπολογισμός Κλασματικού Μέρους

Το module αυτό υλοποιήθηκε στο αρχείο *mant_calc.sv* και σκοπός του είναι να υπολογίζει το κλασματικό μέρος του αριθμού του αποτελέσματος. Οι εισοδοί του (*mant_a*, *mant_b*) είναι τα κλασματικά μέρη των τελεστών συν το κρυφό bit, μεγέθους 24 bit, η διαφορά των εκθετών (*exp_diff*), το *sign_exp_diff* από το *exponent_calc*, καθώς και τα bits προσήμου των τελεστών (*sa*, *sb*). Η έξοδος του είναι το *result_mant* μεγέθους 28 bit.

Αρχικά, σε ένα συνδυαστικό μπλοκ συνδέονται οι εκθέτες εισόδου στα σήματα *small_mant*, *large_mant* ώστε στη συνέχεια να επεξεργαστούν κατάλληλα, ανάλογα με το ποιος τελεστής είναι μεγαλύτερος και ποιος μικρότερος. Για τη σύγκριση αυτή χρησιμοποιείται το *exp_diff* όπου αν είναι 0 θα πρέπει να συγκριθούν τα κλασματικά μέρη, αλλά αν δεν είναι θα χρησιμοποιηθεί το σήμα *sign_exp_diff* που δείχνει ποιος εκθέτης είναι μεγαλύτερος.

Στη συνέχεια, σε διαφορετικό συνδυαστικό μπλοκ γίνεται η δεξιά ολίσθηση του μικρότερου κλασματικού μέρους κατά *exp_diff* ώστε να ευθυγραμμιστεί, και εξάγονται τα guard, round και sticky bits. Στο τέλος, βάσει των προσήμων *sa*, *sb*, γίνεται η πρόσθεση ή αφαίρεση των κλασματικών μερών. Για να μην χαθούν bits κατά τη διάρκεια της ολίσθησης, το κλασματικό μεταφέρεται και επεξεργάζεται μέσω εσωτερικού σήματος των 49 bit. Οι αριθμοί που προστίθενται/αφαιρούνται στο τέλος είναι μεγέθους 27 bits επειδή περιέχουν τα grs bits, και το αποτέλεσμα είναι 28 bits για να αποφευχθεί η απώλεια δεδομένων στην περίπτωση overflow.

2.4 Κανονικοποίηση

Το module της κανονικοποίησης που υλοποιείται στο αρχείο *norm_adder.sv* έχει στόχο την κανονικοποίηση του κλασματικού μέρους μετά την υλοποίηση των προηγούμενων πράξεων, ώστε να έχει μορφή συμβατή με το πρότυπο IEEE-754, δηλαδή να έχει έναν άσσο αριστερά της υποδιαστολής. Στις εισόδους του έχει τον μέγιστο εκθέτη *max_exp*, και το αποτέλεσμα *result_mant* από τον υπολογισμό του κλασματικού μέρους. Για την υλοποίηση αυτού χρησιμοποιείται ένα βοηθητικό module, το *lzc*.

Το *lzc* (Leading Zero Counter) δέχεται στην είσοδο έναν αριθμό μεγέθους 28 bit (*mant_in*). Σκοπός του είναι να μετρήσει και να βγάλει στην 5 bit έξοδο (*zero_count*) τον αριθμό των μηδενικών μέχρι την εύρεση του πρώτου άσσου διασχίζοντας τα bits της εισόδου ξεκινώντας από το MSB. Αυτό σχεδιάστηκε με τη χρήση μιας for loop, μέσω της οποίας ελέγχονται τα bits από το 27ο μέχρι το μηδενικό αυξάνοντας έναν μετρητή κάθε φορά που το τρέχον bit είναι μηδέν. Όταν συναντηθεί ο πρώτος άσσος η λούπα σταματάει.

Με τη χρήση του παραπάνω κυκλώματος μετριούνται τα leading zeros του κλασματικού μέρους που παράχθηκε προηγουμένως. Αρχικά, ελέγχεται αν το MSB του *result_mant* είναι 1, πράγμα που σημαίνει πως έγινε overflow κατά την πρόσθεση των κλασματικών στο module *mant_calc*. Σε αυτήν την περίπτωση πραγματοποιείται δεξιά ολίσθηση κατά 1 στο κλασματικό, και αύξηση του εκθέτη κατά 1. Εδώ, μετά την ολίσθηση το νέο LSB δίνεται από την πράξη OR μεταξύ των δύο LSB πριν την ολίσθηση. Στην περίπτωση που το MSB είναι 0, πραγματοποιείται αριστερή ολίσθηση. Ο αριθμός των ολισθήσεων ισούται με τα

leading zeros μείον 1, εφόσον το MSB θα είναι πάντα 0 σε αυτή την περίπτωση. Ο ίδιος αριθμός αφαιρείται και από τον εκθέτη. Ο εκθέτης αποθηκεύεται σε μεταβλητή μεγέθους 9 bit (*norm_exp*) για την πρόληψη πιθανού overflow/underflow.

2.5 Στρογγυλοποίηση

Το module της στρογγυλοποίησης υλοποιήθηκε στο αρχείο *round_adder.sv* και επεξεργάζεται το κλασματικό μέρος του αποτελέσματος ανάλογα με τον τύπο στρογγυλοποίησης που επιλέγεται. Για καλύτερη διαχείριση δημιουργήθηκε το πακέτο *round_pkg* στο οποίο ορίστηκε το enumeration *round_mode_t* το οποίο κωδικοποιεί τους 5 διαφορετικούς τύπους στρογγυλοποίησης που υποστηρίζονται από το πρότυπο IEEE-754. Το module δέχεται στις εισόδους του τον τύπο στρογγυλοποίησης *round*, το κανονικοποιημένο κλασματικό μέρος *norm_mant*, και το πρόσημο του αποτελέσματος *z_sign*. Αρχικά για ευκολότερη διαχείριση χωρίζεται το *norm_mant* σε *base_mant* (24 bits), *G*, *R*, *S* (3 bits). Με τον υπολογισμό της OR ανάμεσα στα *G*, *R*, *S*, δημιουργείται η έξοδος *inexact_bit* που υποδεικνύει αν η πράξη θα είναι απολύτως ακριβής ή αν υπήρξε απώλεια δεδομένων. Στη συνέχεια, σε ένα συνδυαστικό μπλοκ, ανάλογα με τις τιμές των *round* και *G,R,S*, είτε προστίθεται ένα bit άσσου στο *base_mant* είτε παραμένει όπως είναι, παράγοντας έτσι την έξοδο *round_mant*, μεγέθους 25 bits για την πρόληψη πιθανού overflow.

2.6 Διαχείριση Ειδικών Περιπτώσεων

Στο module *exception_adder* αποφασίζεται ποιο θα είναι το τελικό αποτέλεσμα της εξόδου του αθροιστή, εάν αυτό θα προκύψει από τους υπολογισμούς που έγιναν στα προηγούμενα modules ή αν βρίσκεται σε κάποια από τις ειδικές περιπτώσεις που ορίζει το πρότυπο. Αρχικά δημιουργήθηκε το enumeration *interp_t* το οποίο κωδικοποιεί τις 5 ειδικές περιπτώσεις που υποστηρίζει ο αθροιστής. Στη συνέχεια δημιουργήθηκαν δύο βοηθητικές συναρτήσεις, οι *num_interp* και *z_num*. Η πρώτη δέχεται στην είσοδο έναν αριθμό των 32 bit και επιστρέφει τον τύπο της ειδικής περίπτωσης που ανήκει. Η δεύτερη κάνει την αντίστροφη διαδικασία, δέχεται έναν τύπο ειδικής περίπτωσης και παράγει τον αντίστοιχο αριθμό. Στη συνέχεια, εντός ενός συνδυαστικού μπλοκ, λαμβάνοντας υπόψιν τον τύπο ειδικής περίπτωσης του κάθε τελεστή (και τους συνδυασμούς τους), τα πρόσημά τους, τα flags *overflow*, *underflow* που παράχθηκαν από το Post-Round Normalization και τον τύπο στρογγυλοποίησης *round*, παράγεται το τελικό αποτέλεσμα *result*, και τα flags *zero_f*, *inf_f*, *nan_f*, *tiny_f*, *huge_f*, *inexact_f*, που θα διαμορφώσουν τα τελικά *status* bits εξόδου του κυκλώματος.

3. Επαλήθευση

Στην παρούσα ενότητα περιγράφεται η διαδικασία επαλήθευσης του κυκλώματος, η οποία είναι απαραίτητη για τη διασφάλιση της ορθής λειτουργίας του αθροιστή και της πλήρους συμμόρφωσής του με τις προδιαγραφές του προτύπου IEEE-754. Η διαδικασία αυτή έγινε με τη χρήση προσομοίωσης μέσω ενός testbench. Έγινε σύγκριση των αποτελεσμάτων του κυκλώματος που σχεδιάστηκε με τα αποτελέσματα του μοντέλου αναφοράς της βιβλιοθήκης Hardfloat. Επίσης με τη χρήση SystemVerilog Assertions επαληθεύτηκε η συνέπεια και η λογική ανάθεση των *status* bits εξόδου.

3.1 Προσομοίωση με Testbench

Η επαλήθευση του αθροιστή πραγματοποιήθηκε με την ανάπτυξη ενός αυτοματοποιημένου περιβάλλοντος προσομοίωσης στο αρχείο *tb_adder*. Για τον έλεγχο της ορθότητας των αποτελεσμάτων χρησιμοποιήθηκε ως μοντέλο αναφοράς ο αθροιστής *addRecFN* της βιβλιοθήκης Hardfloat.

Αρχικά, έγινε η σύνδεση των σημάτων εισόδου και εξόδου με το top module του αθροιστή που σχεδιάστηκε. Στη συνέχεια, για την χρησιμοποίηση των ίδιων σημάτων στον αθροιστή *addRecFN*, τα σήματα αυτά επεξεργάζονται μέσω ενός συνδυαστικού μπλοκ ώστε να γίνουν οι απλοποιήσεις των ειδικών περιπτώσεων για να ταιριάζει με το DUT (NaN → Inf, denorm → zero ή MinNormal). Επίσης για να ταιριάζουν στις θύρες του *addRecFN*, και να επιστρέφουν μετά στην ίδια μορφή, επεξεργάζονται από τις συναρτήσεις *fNToRecFN* και *recFNToFN*.

Εφόσον επετεύχθη η τροποποίηση των σημάτων ώστε να είναι συμβατά και στους δύο αθροιστές πρέπει να λυθεί και το πρόβλημα του χρονισμού. Ενώ ο αθροιστής που σχεδιάστηκε είναι ακολουθιακός με καθυστέρηση 2 κύκλων, ο αθροιστής της Hardfloat είναι καθαρά συνδυαστικός. Για αυτό δημιουργήθηκε ένας μηχανισμός καθυστέρησης του αποτελέσματός του. Χρησιμοποιήθηκε ένα *always_ff* μπλοκ το οποίο λειτουργεί ως shift register για τις μεταβλητές του Hardfloat μοντέλου, κι έτσι ευθυγραμμίστηκαν χρονικά τα σήματα.

Στη συνέχεια μέσω ενός ακόμα *always_ff* μπλοκ ελέγχεται η ισότητα των αποτελεσμάτων των δύο αθροιστών. Αν δεν είναι ίσα τότε δημιουργείται ένα μήνυμα με τις πληροφορίες της πράξης στην οποία εντοπίστηκε η αναντιστοιχία.

Εφόσον υπάρχει πλέον ο μηχανισμός ελέγχου των αποτελεσμάτων, πρέπει να υπάρχει και ένας μηχανισμός που παράγει τις εισόδους. Για αυτόν τον σκοπό δημιουργήθηκαν δύο tasks. Το πρώτο παράγει ακολουθία τυχαίων αριθμών για τις εισόδους *a, b*, και *round* με τη χρήση της συνάρτησης *\$urandom*. Το δεύτερο task παράγει 500 συνδυασμούς εισόδων για την κάλυψη όλων των συνδυασμών των ειδικών περιπτώσεων και όλων των τύπων στρογγυλοποίησης.

Σε ένα initial μπλοκ εκτελούνται αυτοί οι δύο τύποι των τεστ, και στο τέλος τυπώνονται τα στατιστικά των αποτελεσμάτων, όπως ο συνολικός αριθμός κάθε είδους τεστ και ο αριθμός των επιτυχημένων τεστ.

3.2 SystemVerilog Assertions

Για την επαλήθευση της σωστής λογικής λειτουργίας και της συνέπειας των εξόδων του DUT χρησιμοποιήθηκαν επίσης SystemVerilog Assertions. Αυτά δημιουργήθηκαν στα αρχεία *test_status_bits.sv* και *test_status_z_combinations.sv*, και έγινε η σύνδεσή τους με το DUT στο testbench με τη χρήση της εντολής *bind*.

Στο module *test_status_bits* υλοποιούνται τα immediate Assertions. Σκοπός τους είναι να διασφαλίσουν ότι ορισμένα status flags είναι αμοιβαίως αποκλειόμενα, καθώς είναι αδύνατο να εμφανιστούν ταυτόχρονα. Ελέγχονται συνολικά 9 συνδυασμοί, όπως *zero_f – inf_f*, *zero_f – nan_f*, *tiny_f – huge_f* κλπ και επίσης ελέγχεται αν τα status bits 6 και 7 είναι πάντα 0 όπως πρέπει, εφόσον το 6 δεν χρησιμοποιείται και το 7 υποδεικνύει διαίρεση με το μηδέν. Για την αποφυγή ψευδών σφαλμάτων κατά την εκκίνηση της προσομοίωσης (χρονική στιγμή 0), όπου τα σήματα βρίσκονται σε άγνωστη κατάσταση, οι έλεγχοι αυτοί θωρακίστηκαν με μια συνθήκη *if*. Έτσι, τα Assertions αξιολογούνται μόνο όταν το σήμα επαναφοράς *resetn* είναι ενεργό και η κατάσταση του status δεν είναι άγνωστη.

Το δεύτερο module *test_status_z_combinations*, περιέχει τα Concurrent Assertions. Εδώ ελέγχεται η συνέπεια των status bits με την έξοδο *result* αλλά και με τις εισόδους που τα παράγαν. Για παράδειγμα, ο πρώτος κανόνας που ελέγχεται είναι ότι εάν είναι υψηλό το σήμα *zero_f* τότε πρέπει στην ίδια ακμή ρολογιού ο εκθέτης του *result* να είναι μηδενικός. Αυτός ο κανόνας εκφράζεται από το πρώτο property και γίνεται *assert* στο *assert_zero_exp*. Ένας άλλος κανόνας που ελέγχεται είναι πως αν είναι υψηλό το σήμα *nan_f*, πρέπει οι τελεστές εισόδου να ήταν άπειροι με αντίθετα πρόσημα. Αυτό γίνεται στο property του *Rule 3* όπου για να “κοιτάξει” δύο κύκλους πριν στο παρελθόν χρησιμοποιείται η συνάρτηση *\$past*. Όταν κάποιος από τους κανόνες αυτούς παραβιάζεται τυπώνεται μήνυμα *error* που ενημερώνει για την παραβίαση.

4. Αποτελέσματα Προσομοίωσης

Η εκτέλεση της προσομοίωσης για την επαλήθευση του κυκλώματος έγινε με το εργαλείο Questa Intel Starter FPGA Edition-64, κι επιβεβαιώθηκε η ορθότητα της σχεδίασης. Όπως φαίνεται στην Εικόνα 1, η προσομοίωση ολοκληρώθηκε επιτυχώς, εκτελώντας 1500 δοκιμές (1000 τυχαίες και 500 ειδικών περιπτώσεων) με 100% ταύτιση των αποτελεσμάτων του σχεδιασμένου αθροιστή με αυτά της Hardfloat.

```
VSIM 2> run -all
# Starting Random Tests...
# Starting Corner Tests...
# =====
#                               SIMULATION STATISTICS
# =====
# Total Tests Executed: 1500
# Random Tests: 1000 / 1000 SUCCESSFUL
# Corner Tests: 500 / 500 SUCCESSFUL
# =====
# ** Note: $stop      : /home/panos/Desktop/sv-fp-adder-ieee754/tb/tb_adder.sv(333)
#   Time: 15055 ns   Iteration: 1   Instance: /tb_adder
# Break in Module tb_adder at /home/panos/Desktop/sv-fp-adder-ieee754/tb/tb_adder.sv line 333
```

VSIM 3>

Εικόνα 1: Στατιστικά προσομοίωσης από το Transcript του Questa

Επίσης, η παρακολούθηση των ελεγκτών SVA καθ' όλη τη διάρκεια της προσομοίωσης απέδειξε τη συνέπεια του συστήματος με τους κανόνες λογικής και χρονισμού. Όπως φαίνεται στην Εικόνα 2, η στήλη Failure Count έχει αριθμό αποτυχιών 0 αποδεικνύοντας ότι δεν υπήρξε καμία χρονική στιγμή κατά την οποία να παραβιάστηκε η αμοιβαία αποκλειστικότητα των *status* flags ή η χρονική τους αλληλουχία με τα δεδομένα εξόδου. Επιπλέον επετεύχθη ο έλεγχος όλων των περιπτώσεων, όπως φαίνεται από το γεγονός ότι στη στήλη Pass Count όλα τα assertions έχουν 1. Αυτό δεν είχε συμβεί με την πρώτη προσπάθεια, καθώς το Pass count για το *assert_huge_val* ήταν 0. Αυτό διορθώθηκε με την προσθήκη επιβολής ενός overflow στο testbench.

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Time
/tb_adder/dut/assert_bits_inst/assert_div_by_zero_bit	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_inf_not_nan	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_inf_not_tiny	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_nan_not_huge	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_nan_not_inexact	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_nan_not_tiny	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_tiny_not_huge	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_unused_bit	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_zero_not_huge	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_zero_not_inf	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_bits_inst/assert_zero_not_nan	Immediate	SVA	on	0	1	-	-	-	-	-
/tb_adder/dut/assert_combos_inst/assert_huge_val	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	-
/tb_adder/dut/assert_combos_inst/assert_inf_exp	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	-
/tb_adder/dut/assert_combos_inst/assert_nan_past	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	-
/tb_adder/dut/assert_combos_inst/assert_zero_exp	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	-

Εικόνα 2: Αναφορά εκτέλεσης των SystemVerilog Assertions